2026“钉耙编程”中国大学生算法设计春季联赛（1）题解

1001 氟化钙

解题思路：

1. **结论推导**
   这是一个取石子游戏，规则是从 $f(a_i, a_j, k)$ 中取 1 到 k 个。
   根据巴什博弈（Bash Game）的结论，先手必败当且仅当石子总数 S 是 $k+1$ 的倍数，即 $S \equiv 0 \pmod{k+1}$。
   $S = a_i \cdot k^{a_i+a_j} + a_j$。注意模 $k+1$ 下，$k \equiv -1$。
   代入得：$a_i \cdot (-1)^{a_i+a_j} + a_j \equiv 0 \pmod{k+1}$。
   结论 （经过分类讨论推导）：
   - 当 k 为偶数时，先手必胜（小远胜）。
   - 当 k 为奇数时，先手必败（小远败）的充要条件是 $a_i + a_j = k+1$ 。

   题目要求统计小远获胜的对数，我们可以用总对数 n^2 减去必败对数。
   必败条件简化： 对于区间 $[l, r]$，若区间最大值 k 为 **奇数** ，且端点满足 $a_i + a_j = k+1$，则该对 (i, j) 导致小远必败。
2. **算法设计**
   枚举最大值： 对于每个位置 p，假设 a_p 是它所在区间的最大值。我们需要找到 a_p 支配的左右边界 $[L_p, R_p]$。
   使用 **单调栈**：
   - L_p：左边第一个 $> a_p$ 的位置（的右侧）。
   - R_p：右边第一个 $\geq a_p$ 的位置（的左侧）。（一边严格一边非严格，是为了防止如果有多个相同的最大值，我们只在最左边那个位置统计一次，避免重复）。

   统计贡献： 对于固定的 p，合法的 i 在 $(L_p, p]$ 范围内，合法的 j 在 $[p, R_p)$ 范围内。我们需要找到满足 $a_i + a_j = a_p + 1$ 的对数。当(i, j)满足条件时，(j, i)也一定满足条件，所以我们只需统计一边再乘2。
   使用 **启发式合并**： 为了保证复杂度，我们遍历区间长度较短的一边，比如 $(L_p, p]$ 长度较短，我们枚举 $[L_p + 1, p]$ 中的所有 i ，并在另一边的区间内查询 $a_j = a_p + 1 - a_i$ 出现的次数，反过来也是一样。查询方法很多，这里我采用的方法是：存储所有元素值和下标对 (a_i, i) 排序后的数组，每次查找在 $[l, r]$ 之间 x 出现的次数时先按值 x 二分找到所有的 (x, i) 所在的区间，再在这段区间中二分 $[l, r]$ 包含的下标个数，即为所求。其他 $O(\log n)$ 内的方法也可（常数超大的主席数我都试过了）。
3. **复杂度**
   单调栈 $O(n)$。
   启发式遍历的总复杂度是 $O(n \log n)$，加上二分查找，总时间复杂度为 $O(n \log^2 n)$。对于 10 组 1×10^5 的数据，完全可以在时限内跑过。

此外，若使用笛卡尔树要注意递归层数可能过深（不过我把内存开很大了应该是能过的）。

1002 数字王国的警报密码

题意

构造一个长度为 n 的严格递增正整数序列 $a_1, a_2, \ldots, a_n$，使任意连续子段和都不为 k 的倍数，并且字典序最小。

思路

设前缀和：

$$s_i = a_1 + a_2 + \cdots + a_i$$

任意子段和为：

$$s_r - s_{l-1}$$

要求：

$$(s_r - s_{l-1}) \bmod k \neq 0$$

等价于：

$$s_r \bmod k \neq s_{l-1} \bmod k$$

结论：所有前缀和模 k 必须互不相同。

无解情况

前缀和共有 $n+1$ 个（包括 $s_0 = 0$），而模 k 只有 k 种取值：

$$n + 1 \leq k \Rightarrow n < k$$

若 $n \geq k$，则无解。

构造方法

当 $n < k$ 时，贪心构造：

- 维护当前前缀和 s
- 维护已出现的 $(s \bmod k)$

从左到右枚举，每次取最小的 $a_i > a_{i-1}$，满足：

$$(s + a_i) \bmod k$$

未出现过

然后更新：

- $s \leftarrow s + a_i$
- 标记该模数

复杂度

时间复杂度：$O(\log k)$ （实际低于这个值）
空间复杂度：$O(k)$

总结

本题关键在于：

- 将“子段和限制”转化为“前缀和 mod k 不重复”
- 在此基础上用贪心构造字典序最小解

1003 回文串

题意

给定两个只包含小写字母的字符串S和T，长度均为n，你可以在S和T中分别选取子串S'和T'，并将T'拼接在S'后面组成新字符串。

问有多少种选法，使得组成的新字符串是个**回文串**。

题解

对于组成的新字符回文串，可以将其看成由三部分ABA'组成，其中$A = reverse(A')$，B为回文串。

这样可以得到一个n^3的做法，L_s和R_s分别表示A的左右端点，L_t和R_t 分别表示A' 的左右端点，在S中枚举L_s，在T中枚举R_t，因为$A = reverse(A')$，所以R_s和L_t是同步的，枚举其中一个即可。因此我们现在枚举了四元组(L_s, R_s, L_t, R_t)，若$S[L_s, R_s] = reverse(T[L_t, R_t])$，则这就是一个新回文串。回到最开始的分解$ABA'$，我们现在枚举了$A$和$A'$，还需要加上中间$B$的个数，也就是$S$中以$R_s + 1$为左端点的回文串个数和$T$中以$L_t - 1$为右端点的回文串个数，这个用马拉车提前预处理即可。

考虑优化，容易发现枚举R_s 和L_t使得$S[L_s, R_s] = reverse(T[L_t, R_t])$的过程，满足单调性，可以用二分枚举加哈希$check$，这样复杂度降至$O(n^2 logn)$，写的比较优秀可能可以通过此题。

可以接着优化，先将T反转一下，定义$f[i][j]$表示S_i和T_j往前有多少个字符是相等的，ps_i和pt_i分别表示S中以i为左端点的回文串个数和T中以i为左端点的回文串个数。Ans即为$\sum f[i][j] \times (ps_i + pt_j + 1)$。其中$ps_i$和$pt_i$和之前一样用马拉车预处理，$f$数组根据定义也很好推，若$S_i = T_j$，$f[i][j] = f[i-1][j-1] + 1$；若$S_i \neq T_j$，$f[i][j] = 0$。$n^2$可通过此题。

1004 炼金术士的配方

思路

先定义一个数 x 的状态 $s(x)$：

若

$$x = \prod p_i^{e_i},$$

则

$$s(x) = \{\, p_i \mid e_i \equiv 1 \pmod 2 \,\}.$$

也就是说，$s(x)$ 只保留质因数分解中指数为奇数的那些质数。题目中定义的“核心本质”，其实就是把 $s(x)$ 中所有质数乘起来。

例如：

- $12 = 2^2 \cdot 3$，所以 $s(12) = \{3\}$，核心本质为 3；
- $90 = 2 \cdot 3^2 \cdot 5$，所以 $s(90) = \{2, 5\}$，核心本质为 10。

对于两个正整数 u, v，它们相乘时每个质因子的次数会相加，因此只看奇偶性时有：

$$s(uv) = s(u) \oplus s(v)$$

这里的 $\oplus$ 表示按位异或意义下的合并：某个质因子出现两次会抵消，出现一次会保留。

于是一个区间 $[l, r]$ 的乘积状态为：

$$s\left(\prod_{i=l}^{r} a_i\right) = s(a_l) \oplus s(a_{l+1}) \oplus \cdots \oplus s(a_r)$$

设前缀状态为：

$$pre_i = s(a_1) \oplus s(a_2) \oplus \cdots \oplus s(a_i), \qquad pre_0 = 0$$

则区间 $[l, r]$ 的状态就是：

$$pre_r \oplus pre_{l-1}$$

设目标状态为 $T = s(X)$，那么题目要求满足：

$$pre_r \oplus pre_{l-1} = T$$

移项得：

$$pre_{l-1} = pre_r \oplus T$$

所以，枚举右端点 r 时，只需要统计前面有多少个前缀状态等于 $pre_r \oplus T$。

这就是经典的 **前缀异或 + 哈希表计数**。

状态表示

由于状态本质上是“若干质因子的奇偶集合”，直接存集合不方便。

可以给每个质数 p 随机分配一个 64 位无符号整数 R_p，定义一个数 x 的哈希状态为：

$$h(x) = \bigoplus_{p \in s(x)} R_p$$

这样就能用一个 `unsigned long long` 表示状态，并且仍然满足：

$$h(uv) = h(u) \oplus h(v)$$

于是原问题就转化为统计有多少对 (l, r) 满足：

$$pre_r \oplus pre_{l-1} = h(X)$$

做法

1. 用筛法预处理出每个 v 的状态哈希 `hsh[v]`；
2. 令 `tar = hsh[X]`；
3. 从左到右扫描数组，维护当前前缀状态 `cur`；
4. 若某个左侧前缀状态 `need = cur ^ tar` 出现过 `cnt` 次，那么这些位置都能与当前右端点组成合法区间；
5. 用哈希表统计前缀状态出现次数即可。

正确性说明

由定义，$s(x)$ 只记录各质因子指数的奇偶性。

而乘法会使指数相加，所以对模 2 意义下的状态来说：

$$s(uv) = s(u) \oplus s(v)$$

因此区间 $[l, r]$ 的状态可写为：

$$pre_r \oplus pre_{l-1}$$

题目要求该状态等于 $T = s(X)$，即：

$$pre_r \oplus pre_{l-1} = T$$

等价于：

$$pre_{l-1} = pre_r \oplus T$$

所以对于每个右端点 r，答案增加此前出现过的状态 `pre_r ^ T` 的个数。

算法恰好不重不漏地统计了所有合法区间，因此正确。

复杂度分析

设值域上界为 M。

预处理状态哈希复杂度约为：

$$O(M \log \log M)$$

扫描数组并统计答案复杂度为：

$$O(N)$$

总复杂度为：

$$O(M \log \log M + N)$$

空间复杂度为：

$$O(M + N)$$

1005 大户爱的开根

样例 `64 3` 为一种题目所描述的精度丢失情况（视编译器差异可能会有所不同）。

本题做法很多，最容易想到的做法是每次枚举答案然后乘 K 次，若 $i^K \le N$ 且 $(i+1)^K > N$，则 i 为答案。需要特判 $1^K = 1$，以及乘到超过 N 的时候就不需要继续乘了，因为没有保证 $\sum K$ 的大小。

此做法的复杂度是 $O(T\sqrt{N})$。当然你只要能通过这道题，你的做法就是好的，不需要拘泥于我的做法。

1006 巧克力工厂

考虑贪心，将巧克力按加工完成时间从小到大排序，能吃就吃，不能吃就等，别忘了开 long long。

时间复杂度 $O(n \log n)$。

1007 流沙

如果这只问整棵树的答案（即只求 $f(1)$），可以使用二分答案 X，然后用 $O(N)$ 树形 DP 或贪心验证，复杂度是 $O(N\log(\sum A_i))$。

仔细思考沙子“只能向父节点移动”的物理意义：对于任何一个节点 v，它子树内的沙子总量 S_v 是固定的，且永远无法突破子树流到外面去。
因此，不管怎么移动，子树 v 内部的平均沙子数最多就是 $\lfloor S_v/C_v \rfloor$（C_v 为子树节点数）。由于沙子可以自由向根汇聚，只要所有的子树内部的上限不卡脖子，这个平均值就一定能达到。
由此得出转移方程：

$$f(u) = \min\left(\left\lfloor \frac{S_u}{C_u} \right\rfloor, \min_{v \in children(u)} f(v)\right)$$

即任何一个节点的答案，仅仅受限于它“自身的平均值”和“它所有子节点的答案”。

1008 大户爱的gcd

对于每个位置 a_i，它的值只跟包含 i 的给定关系中的 g 有关。如果有 k 个关系包含 i，令这 k 个关系的 g 为 $g_1, g_2, \ldots, g_k$，则 a_i 一定是 $\operatorname{lcm}(g_1, g_2, \ldots, g_k)$ 的正整数倍。对于每个 a_i 记录一个 b_i 维护 $\operatorname{lcm}(g_1, g_2, \ldots, g_k)$ 的值，则对于一个询问 `x y`，答案为 $\gcd(b_x, b_y)$。因为题目保证 $g \le 30$，所以 b_i 在 long long 范围内。

- lcm 为最小公倍数，gcd 为最大公因数，gcd 可用欧几里得算法求得，$\operatorname{lcm}(x, y) = \frac{xy}{\gcd(x,y)}$。

1009 力的平衡

题意本质

给定若干二维整数向量，每个向量可以使用任意次。

要求找到一个**最短的非空序列**，使这些向量之和恰好为 $(0,0)$；若不存在，输出 `-1`。

核心结论

乍看之下，因为每种向量都能无限次使用，状态空间是无限的，无法直接搜索。

关键是用 **二维 Steinitz 引理** 把“无限状态”压成“有限状态”。

Steinitz 引理在本题中的用法

设
$B = [-100, 100]^2$
题目保证每个向量都属于 B。

二维 Steinitz 引理给出：

> 若一组二维向量的和为 0，且每个向量都落在关于原点中心对称的凸集 B 中，
> 那么存在一种重排，使得所有前缀和都落在 $2B$ 中。

这里
$2B = [-200, 200]^2$
因此立刻得到：

> 如果本题有解，那么一定存在一组同样长度的解，使得其所有前缀和始终位于
> $[-200, 200]^2$
> 内。

这一步说明：**最优解一定可以在一个有限方框内找到。**

图论建模

既然最优解的所有前缀和都不会离开 $[-200, 200]^2$，就可以在这个有限网格上建图。

- 一个状态表示当前向量和 (x, y)
- 从 (x, y) 选择一个向量 (F_{x_i}, F_{y_i})，转移到
  $(x + F_{x_i},\ y + F_{y_i})$
- 只保留仍在 $[-200, 200]^2$ 内的转移

于是问题变成：

> 在这个有限图中，求从 $(0,0)$ 出发、经过至少一条边后再次回到 $(0,0)$ 的最短路长度。

由于每次转移代价都为 1，直接 BFS 即可。

算法做法

1. 特判

若输入中存在向量 $(0,0)$，那么用一次就已经平衡，答案直接为 `1`。

2. BFS 状态

令 `dist[x][y]` 表示从原点到状态 (x,y) 的最少步数。

由于坐标有负数，可整体平移 `+200` 存数组。

3. 转移

从当前状态 (x,y)，枚举每个向量 (dx,dy)：

- 计算下一状态 $(nx,ny)=(x+dx,y+dy)$
- 若越界，跳过
- 若 $(nx,ny)=(0,0)$，则用 `dist[x][y] + 1` 更新答案
- 否则若未访问过，就入队

注意：

- 原点只作为起点入队一次
- 之后"回到原点"不再入队，而是直接更新答案

这样就保证求到的是"最短非空闭合路径"。

正确性说明

性质 1: 若本题存在解，则存在一组最优解，其所有前缀和都落在 $[-200,200]^2$ 中。

原因： 直接由上面的二维 Steinitz 引理推出。

性质 2: BFS 不会漏掉最优解。

原因： 由性质 1，某组最优解对应的一整条路径都在搜索方框内；而 BFS 会遍历这个有限图中所有从原点可达的状态，因此一定能扫描到这条最优闭环路径。

性质 3: BFS 求出的答案就是最少使用次数。

原因： 图中每条边代价都为 1，BFS 求得的是最短边数；而每条边恰好对应一次使用发生器，所以得到的正是最少操作次数。

综上，算法正确。

复杂度分析

方框大小固定为
$401\times401=160801$
设去重后的向量数为 k。

- 状态数：$O(160801)$
- 每个状态最多枚举 k 个转移

故单组时间复杂度为
$O(160801 \cdot k)$
空间复杂度为
$O(160801)$

在本题数据范围下完全可行。

1010 MEX

不难发现在排列中有：$MEX_P[l,r] = min(a_1,\ldots,a_{l-1},a_{r+1},\ldots,a_n)$ ，故所生成的 B 序列一定是单调不增的。

考虑从前往后放数。可以发现 B 序列将原序列 A 划分成了一段一段，第 i 段除两端外为 $[x_{i-1},x_i)$，第 i 段可以放数的范围为：$[y_i,n-1]$，并且 y_i 是必须放置在这一段范围内的，而由于 B 序列是单调不增的，故较后面的数的约束宽于前面的数，用掉的数是严格包含在第 i 段的范围内的，故可以直接在范围里相减，计算出在每一段放数的方案数，再对两端讨论一下即可。

同时对 $y_i = y_{i-1}$ 的情况，由于 y_{i-1} 这个数在前一段已经放置过，故不用再考虑其在这一段的放置。而方案数为 0 的情况即是 $b_1 \neq n$ ，B 序列不单调不增以及每一段放的数的数量小于位置数。